

AI WeatherWise - Deliver intelligent weather forecasts and insights.



PROJECT TITLE: **AI Weather Wise**

TEAM ID:

TEAM LEADER: Manju.: 24csmanju055@gmail.com

TEAM MEMBERS:

Madhumitha.B 24csmadhumitha051@gmail.com

Mageshwari.V: 24csmageshwari053@gmail.com

Malathy.v: 24csmalathy055@gmail.com

PROJECT OVERVIEW:

AI WeatherWise is a smart weather application developed using **Node.js, Express.js, MongoDB, and Google Gemini AI**. It provides real-time weather information, simple AI-generated summaries, and personalized recommendations for activities and clothing. The system also supports secure user authentication and favorite location management. The system also allows users to **manage favorite locations securely** and access weather information conveniently through a simple and user-friendly platform.

MAIN OBJECTIVES:

- To provide **real-time weather information**.
- To generate **AI-based weather summaries and recommendations**.
- To help users **manage favorite cities and locations**.
- To provide **personalized activity and clothing suggestions**.
- To ensure **secure user authentication and data storage**.
- To create a **simple and user-friendly weather service**.



SCENARIO-BASED CASE STUDY:

BACKGROUND: Alex travels frequently and needs quick access to weather information and personalized suggestions.

PROBLEM: Managing multiple cities and understanding raw weather data can be difficult, and users may lack personalized recommendations.

SOLUTION: AI WeatherWise combines real-time weather data with Gemini AI to provide easy-to-understand summaries and personalized recommendations.

OUTCOME: The system helps users make better weather-related decisions while securely managing their preferences and favorite locations.



SOFTWARE REQUIREMENTS:

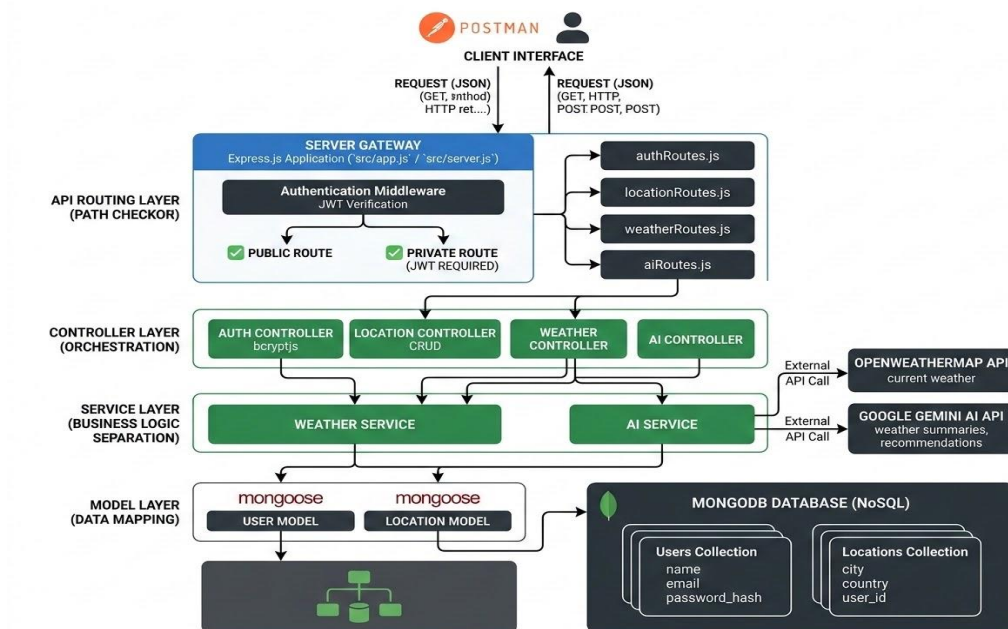
- OS: Windows 10/11, macOS, or Linux
- Node.js & npm: For backend development and package management
- MongoDB & Mongoose: For data storage and management
- Gemini AI SDK: For AI-based weather insights
- Postman: For API testing
- VS Code: For coding and project development

HARDWARE REQUIREMENTS:

- **Processor:** Intel i5 / AMD Ryzen 5 or equivalent
- **RAM:** 8 GB minimum
- **Storage:** 1 GB free space
- **Internet:** Required for weather and AI services .

1:PROJECT REQUIREMENTS:

The AI WeatherWise system is designed using a structured MVC architecture that separates application logic, data handling, and user requests. The Model handles



user accounts and saved locations using MongoDB Mongoose Management.

2. Architecture Flow

Client → API Gateway → Weather Service

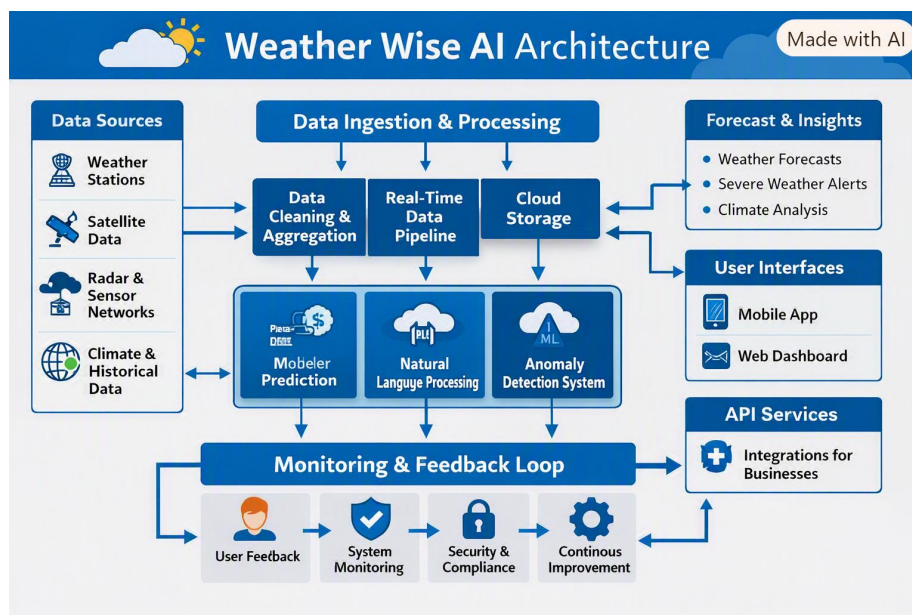
Weather Service checks `isMock`

If true → Mock Data Service

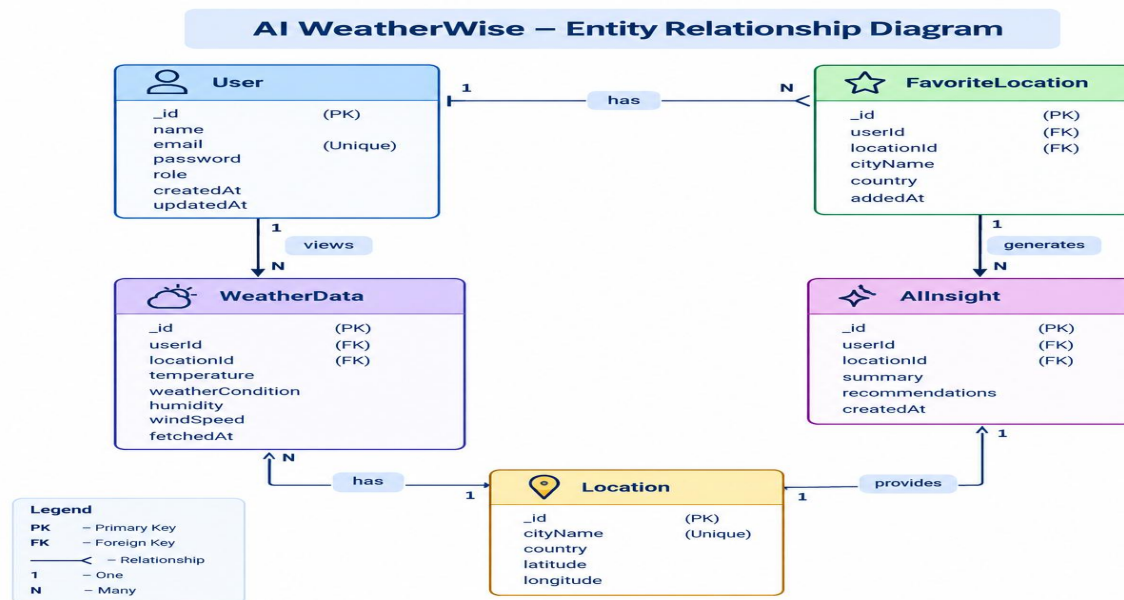
If false → Real Data Service

Data Processing Layer formats response

Response sent back to client



It visually maps out the system from **data sources** (weather stations, satellites, radar, historical climate data) through **data ingestion & processing**, into **AI/ML models** (prediction, NLP, anomaly detection), and finally to **forecast outputs** like mobile apps, dashboards, and API services. At the base, a **monitoring & feedback loop** ensures continuous improvement, security, and reliability.



USER ENTITIES:

- **_id:** Primary Key – ObjectId
- **name:** String, Required
- **email:** String, Required, Unique
- **password:** String, Hashed, Required
- **role:** String, Default: "user"

LOCATION ENTITIES:

- **_id:** Primary Key – ObjectId
- **user:** Foreign Key referencing User Collection
- **city:** String, Required
- **country:** String, Required
- **createdAt:** Date, Default: Now

3.DATA RELATIONSHIPS:

A single registered user can store multiple favorite locations, creating a one-to-many relationship between User and Location.

4.MAIN FUNCTIONS:

- **User Security:** JWT authentication and encrypted passwords protect accounts.
- **Location Management:** Users can save and manage favorite cities.

- **Weather Access:** Current weather details are retrieved for selected locations.
- **AI Assistance:** Gemini provides weather summaries and personalized suggestions.
- **System Protection:** Validation and request sanitization improve API security.

5.USER RESPONSIBILITIES:

Admin: Manages users, monitors API services, and checks system performance.

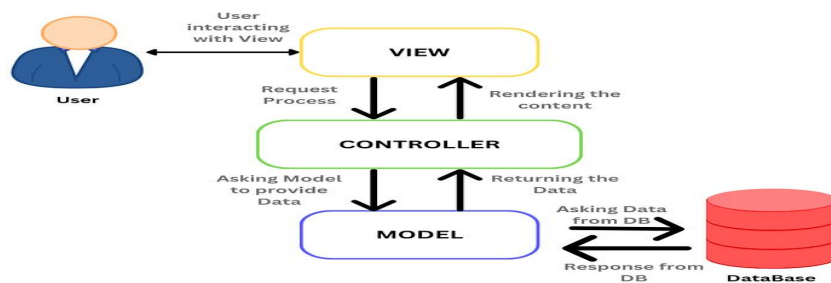
Registered User: Maintains favorite cities, views weather information, and receives AI-based recommendations.

AI WeatherWise API User Workflow



6.MVC ARCHITECTURE PATTERN:

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



AI WEATHER WISE– BACKEND ARCHITECTURE:

The AI WeatherWise backend is organized into separate modules to keep the system clear and manageable. The **Model** module defines MongoDB collections and validation rules using Mongoose, while the **Controller** module handles requests, executes application logic, and returns JSON responses. Since the project works as a RESTful backend without a graphical interface, the **Routing** module connects API requests to the required controllers.

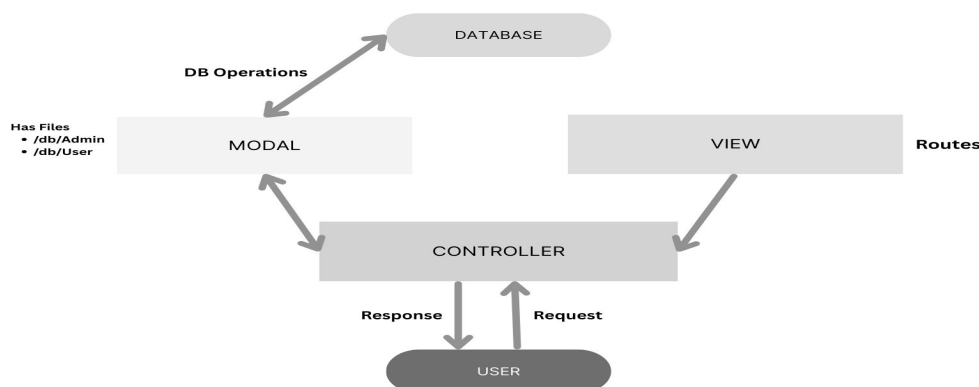
AI WEATHER WISE – PROJECT CONFIGURATION:

The project is initialized as a **Node.js application** and developed in **Visual Studio Code**. Required packages such as Express, Mongoose, bcryptjs, JWT, CORS, dotenv, and Gemini AI are added through npm to support server operations, database connectivity, authentication, and AI services. Development tools such as Nodemon are also included for automatic server restarting during coding.

AI WEATHER WISE – PROJECT STRUCTURE:

The application includes essential files such as **index.js**, **package.json**, **.env**, and **.gitignore**, along with separate folders for models, controllers, routes, middleware, and services. This organized structure improves code management, simplifies maintenance, and allows new weather and AI features to be added easily.

PROJECT SETUP AND CONFIGURATION:



CREATING THE PROJECT FOLDER:

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

PROJECT INITIALIZATION:

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer  
@google/genai
```

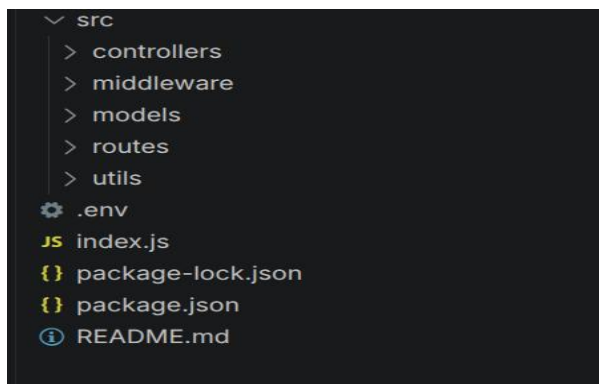
Install Nodemon for development:

```
npm install --save-dev nodemon
```

CREATE THE FOLLOWING FILES:

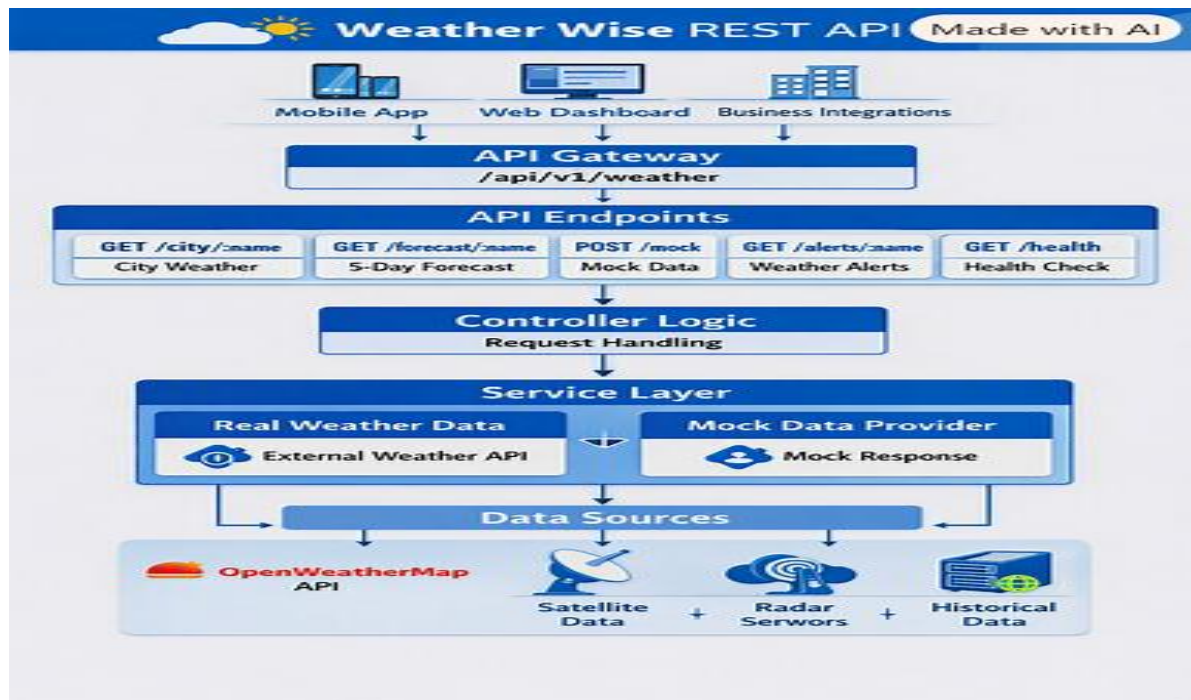
- index.js-.env
- package.json
- .gitignore

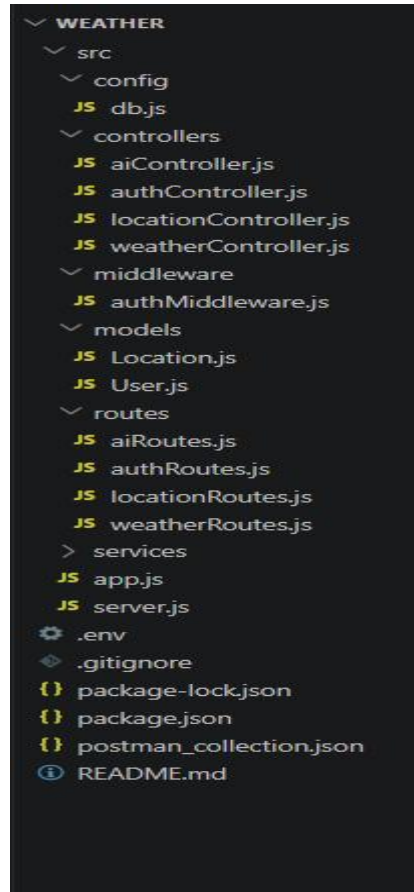
CREATE THE FOLLOWIING FOLDER STRUCTURE:



BACKEND DEVELOPMENT:

The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.





CORE SYSTEM SCRIPT IMPLEMENTATION:

1. APPLICATION ENTRY POINT (**index.js**):

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

DATABASE CONFIGURATION:

API TESTING DEMO

1. CONFIGURE MONGODB:

- Install Mongoose In your Node.js project directory, run:npm install mongoose
- Create database connection.
- Create Schemas & Models.

MONGOOSE SCHEMA DEFINITIONS:

Location Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

User schema:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

DB CONNECTION:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

PROJECT EXECUTION:

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

RUNNING BACKEND SERVER:

Install all required dependencies:

```
npm install
```

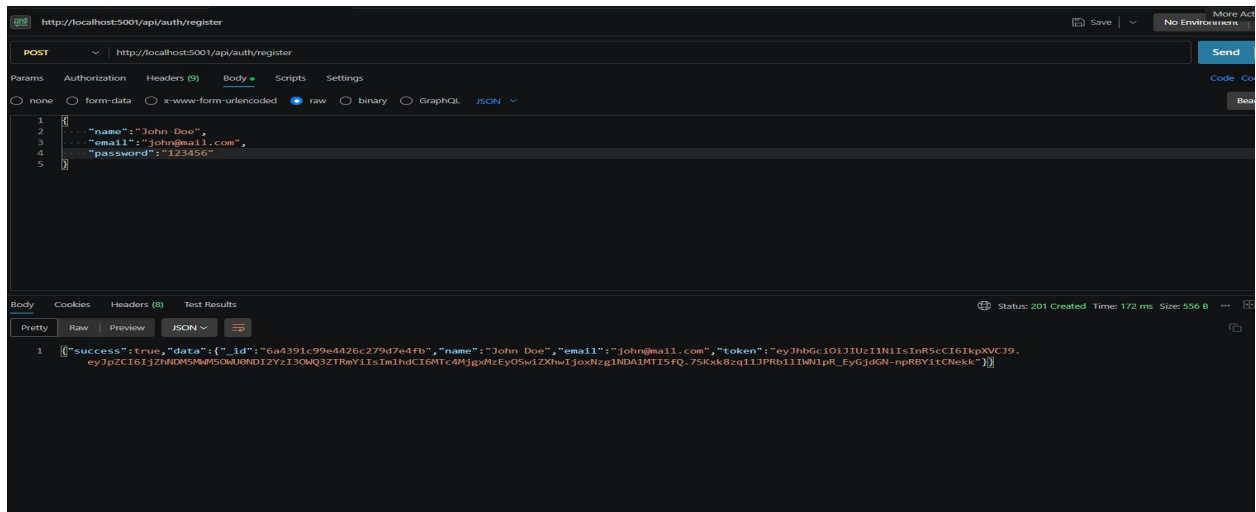
Start the development server:

```
npm start
```

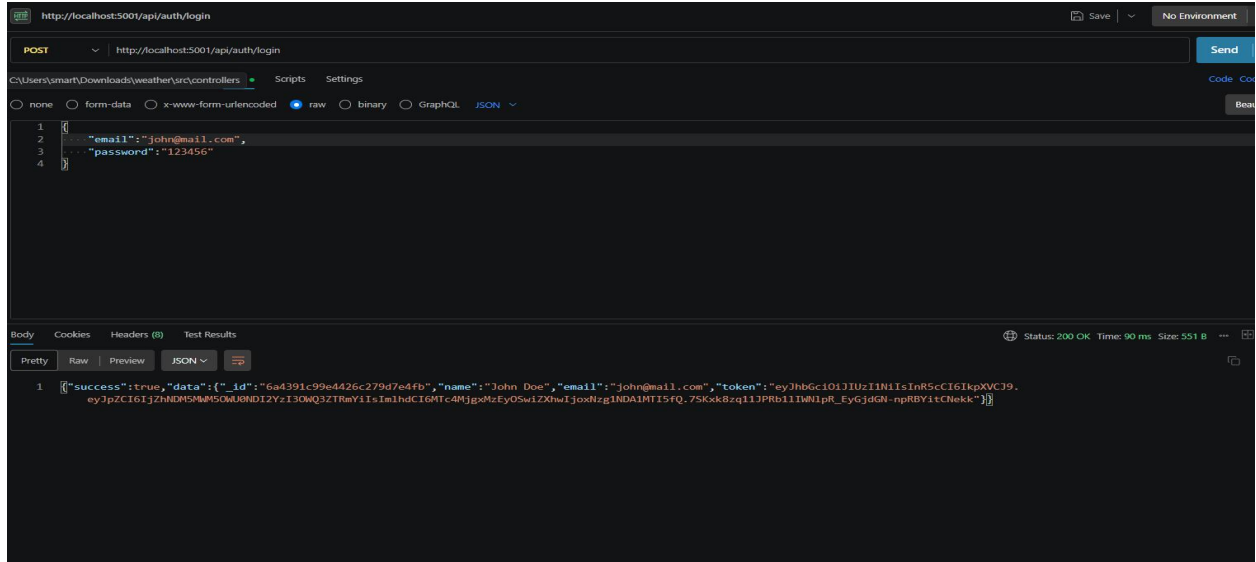
After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API TESTING:

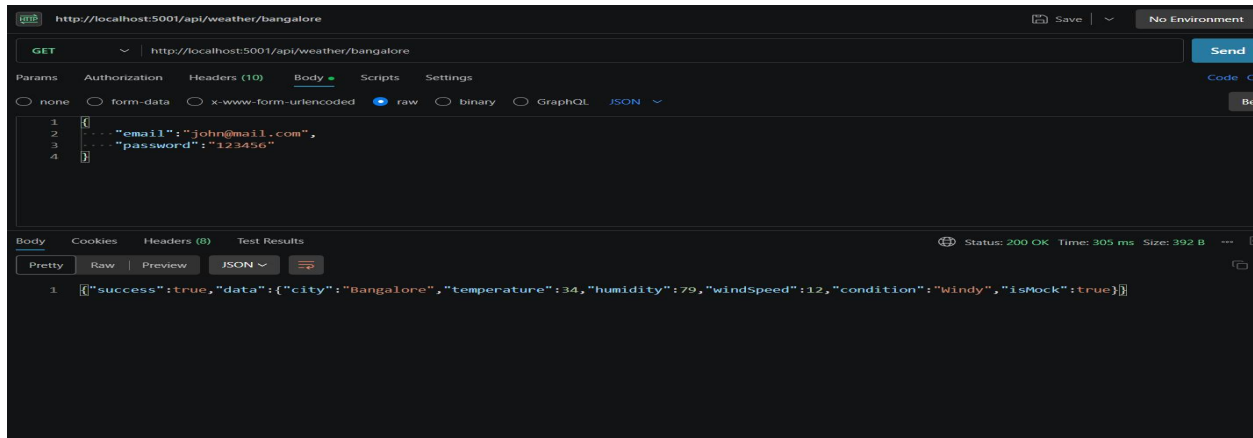
REGISTER API:



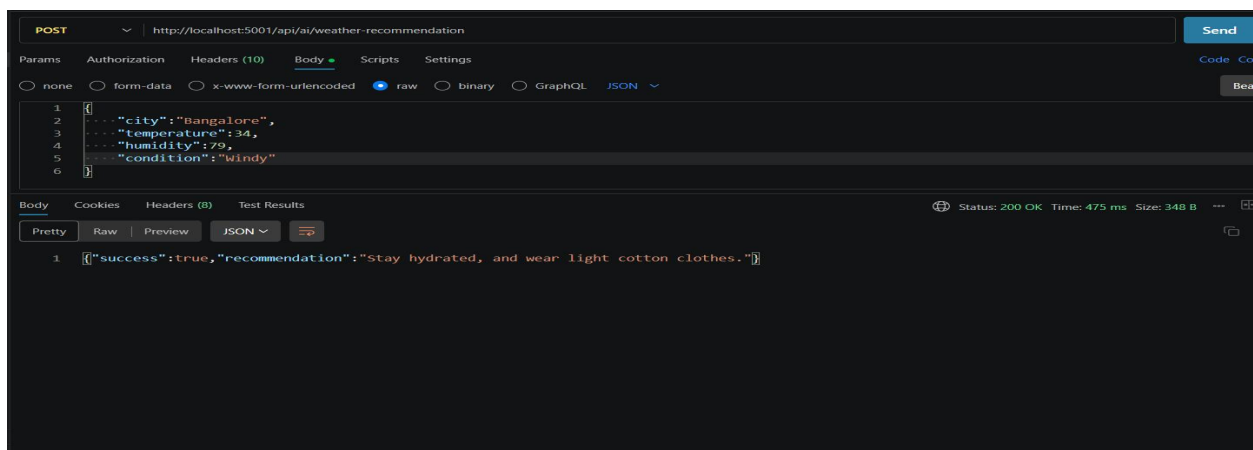
LOGIN API:



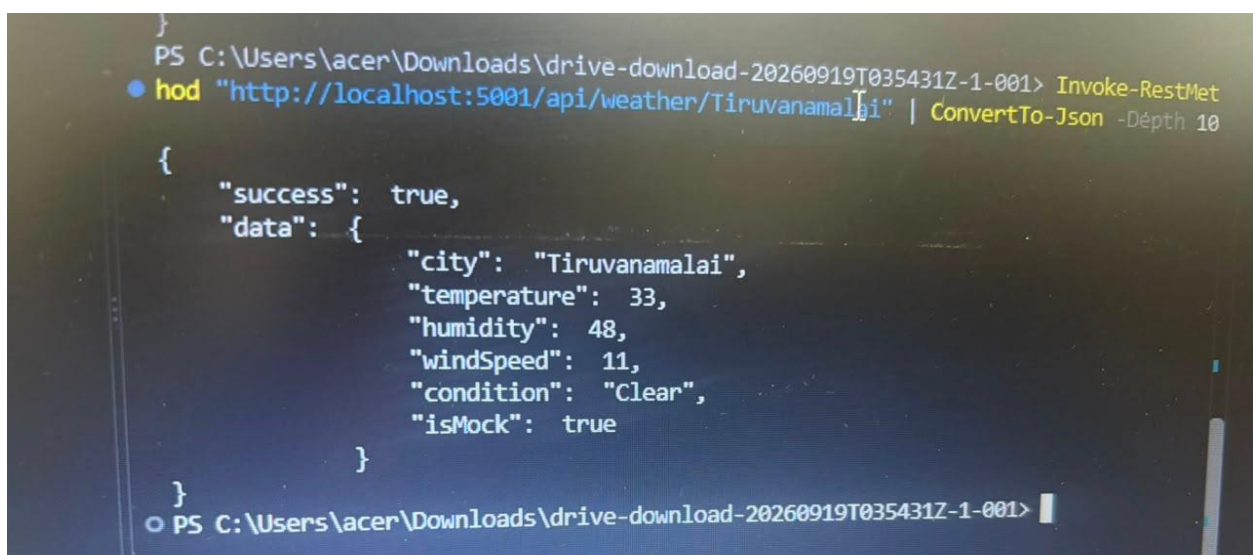
WEATHER API:



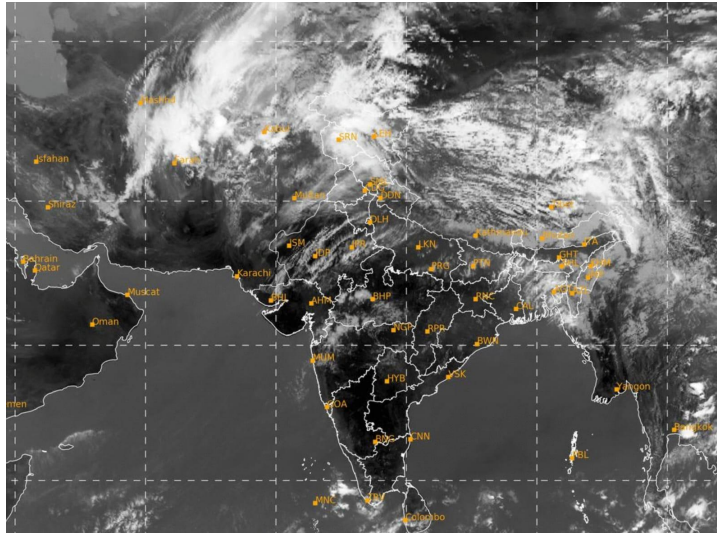
RECOMMENDATION API:



OUTPUT:



THANK YOU..!



BY OUR TEAM CREW :

MANJU.M

MADHUMITHA.B

MAGESHWARI.V

MALATHY.V

API testing video link: [API testing.mp4](#)

Demo Video Link:

<https://drive.google.com/file/d/19heVm1V0nV7B8IU7Oau-yJkpBACtTyuT/view?usp=drivesdk>

CodeDrive: [Code](#)